

Using a language model you did not train

A small pretrained transformer runs on a laptop CPU. It does one thing: given some tokens, it produces a probability for every possible next token. Today you learn how that single ability becomes generated text, why the same prompt gives different answers, and how thirty sentences of fine-tuning change its voice.

Unit 5 · Language

80 minutes

Lab 17 · generate, then fine-tune

distilgpt2 · 82 million parameters

Next token

Temperature

Fine-tuning

Its limits

Lab 17

Before Tuesday

PART 1 · 15 MINUTES

One ability: the next token

1

A causal language model reads a sequence of tokens and outputs, for the next position, a score — a **logit** — for each of the 50,257 tokens in its vocabulary. A softmax turns the scores into probabilities. That is all it computes. Everything else is what we do with those probabilities.

GENERATION IS A LOOP

Predict a distribution for the next token. **Choose** one. Append it to the input. Repeat. A paragraph is dozens of these choices in a row, each conditioned on every earlier choice — which is why one odd early choice can send the whole text somewhere strange.

It is trained the same way: on ordinary text, at every position, the loss is the cross-entropy between the predicted distribution and the token that actually came next. Session 11's loss, with 50,257 classes instead of 2, applied billions of times. No labels are needed beyond the text itself.

Sampling and temperature

Why the same prompt gives different text each time.

2

Always picking the most probable token is **greedy decoding**; it tends to repeat itself. Instead we *sample* from the distribution. **Temperature** T divides the logits before the softmax, sharpening or flattening the distribution. Three candidate tokens with logits 2.0, 1.0 and 0.1:

Temperature	Token A	Token B	Token C	Effect
0.5	0.864	0.117	0.019	sharper: almost always A — safe, repetitive
1.0	0.659	0.242	0.099	the model's own distribution
2.0	0.502	0.304	0.194	flatter: surprising, often incoherent

TEMPERATURE CHANGES HOW IT SAMPLES, NOT WHAT IT KNOWS

Every probability in that table comes from the same logits. Turning temperature down does not make the model more correct — only more predictable. A false statement the model rates as likely is produced *more* reliably at low temperature, not less. Top-k and top-p sampling are further ways of trimming the unlikely tail before choosing.



Seeds make it reproducible

Sampling uses a random generator. Fix the seed, the prompt and the temperature, and the text is the same every run. That is how Lab 17 makes a before-and-after comparison fair.

Fine-tuning: transfer learning, one modality across

The same move as session 23, on text.

3

distilgpt2 has 82 million parameters, pretrained on a large web-text corpus. Lab 17 continues training *all* of them, briefly, on a thirty-sentence corpus in which every sentence begins “In CSC 4315,” — predict first, the split happens before scaling, a loss of 0.693 means nothing learned. Four epochs, a small learning rate, the same next-token loss.

the fine-tuning loop in Lab 17

PYTHON

```
for epoch in range(4):
    for (batch,) in loader:
        out = model(input_ids=batch, labels=batch)  # next-token cross-entropy, computed
inside
        out.loss.backward()                        # session 14, via autograd
        opt.step(); opt.zero_grad()                # session 18
```

Then generate from the same probe — same prompt, seed and temperature — before and after. The difference is the parameters, and only the parameters. After fine-tuning the text borrows the corpus's phrasing and topics: the voice has changed.

WHAT FINE-TUNING SHOWS, AND WHAT IT DOES NOT

It shows that a small, targeted dataset can shift a large model's behaviour cheaply. It does not show understanding: the model has learned which tokens follow which in thirty sentences. Fine-tune for too long and it recites the corpus word for word — **session 22's overfitting**, now audible.



The same responsibilities as unit 4

A fine-tuning corpus is training data. It needs a held-out set to detect memorising, it carries the biases of whoever wrote it, and anything private in it can come back out in generated text.

Why this model writes nonsense

4

Lab 17's model will produce fluent, confident, frequently absurd text. It has 82 million parameters, was trained on a fraction of the text frontier models see, and was never taught to follow instructions. Tuesday explains precisely which of those differences turns this into a system that can help write your code.

Generate, then fine-tune

5

Lab 17 — `CSC4315_Lab17_Generate_Then_Fine_Tune` . A temperature table from three settings and two seeds, the provided fine-tuning run, and a controlled before-and-after comparison. The model, corpus and script are provided.

Before Tuesday

1. **Finish Lab 17.** Unit 5 ends here.
2. **Bring one frontier model** open in a browser tab on Tuesday, and one task from your own studies you would like it to do well.



Session 27 — Tuesday 08 December, 16:00

From small models to frontier models, and prompting as engineering.